Evolution 2 · Lecture

Patterns and Orchestration Frameworks

Workflows versus agents; the five patterns; every framework is a theory of who owns the loop; ADR-1

SURVEY | BUILD | GOVERN | DEFEND

CIDM 6333 · Agentic Systems Design and Development
Survey: LangChain → LangGraph · Pydantic AI · AutoGen / CrewAI / MetaGPT · the vendor SDKs · Build: the loop refactored into a named pattern; ADR-1

Sixty minutes, five moves

0:00–0:02

The run of show

1	0:00	**Framing**	Every framework is a theory of who owns the loop; workflows versus agents, again
2	0:10	**Spine**	Five workflow patterns and the agent; keep it simple; refactor E1's loop
3	0:30	**Teardowns**	LangChain → LangGraph; Pydantic AI; AutoGen / CrewAI / MetaGPT; OpenAI Agents SDK, Claude Agent SDK, Google ADK
4	0:45	**Reference design**	ADR-0.0.2 — 'popularity is not rationale'; the decomposition scorecard; three forcing functions
5	0:52	**Evaluation lens**	What a strong Sortie 2 and a strong ADR-1 look like; Flight Card 2 brief

Every framework is a theory of who owns the loop

0:02–0:10

The real subject is control, not the API

Frameworks change monthly. Each encodes a theory of where control lives: your code, the model, the vendor's harness, or a graph you draw. You choose and defend that theory.

Workflows versus agents (Anthropic, Dec 2024)

Workflow: the model runs along code paths you wrote. Agent: the model directs its own control flow. Most production 'agents' are workflows, and should be.

Tonight's claim, which Sortie 2 will try to disprove
Same task, two patterns, identical tool traces, and the cost or quality delta ADR-1 predicted. No predicted delta in the evidence: the ADR was rationalization.

The five workflow patterns and the agent

0:10–0:18

Anthropic, 'Building Effective Agents' — the catalog you refactor into

Prompt chaining

Fixed sequence; each step's output is the next step's input; optional gate between steps.

When: Decomposable task; accuracy over latency

Routing

Classify the input, dispatch to a specialized path.

When: Distinct input classes that deserve distinct prompts or models

Parallelization

Sectioning (split, run, merge) or voting (same task N times, aggregate).

When: Independent subtasks; confidence from multiple samples

Orchestrator–workers

A central model decomposes the task, delegates, and synthesizes.

When: Subtasks unknown in advance; large edits across files

Evaluator–optimizer

One model produces, another critiques, loop until criteria met.

When: Clear evaluation criteria; iteration measurably helps

The agent

Open loop: model plans, acts on tools, observes, continues until done or halted.

When: Open-ended problems; the path cannot be scripted; trust and budget in place

Refactoring E1's loop without touching the tools

0:18–0:25

The trace must stay identical; only control moves

E1: the open loop

model(messages) → Step
  answer? halt
  tool? call → observe
repeat under BUDGET

Control: the model, bounded by code.
Trace: T1 tool A, T2 tool B, T3 answer.

As orchestrator–workers

orchestrator(messages) → [subtask$_1$, subtask$_2$]
worker(subtask$_1$) → calls tool A
worker(subtask$_2$) → calls tool B
synthesize → answer

Control: a planner plus bounded workers, each with its own budget.
Trace: same tools, same order, or the refactor changed behavior.

As evaluator–optimizer

producer(messages) → draft (tools A, B)
evaluator(draft, criteria) → PASS | revise
loop ≤ K rounds → answer

Control: a critic gates the halt; the halt condition is a criterion.
Trace: same tools, plus evaluator turns you can count.

Card 2's discipline: hold the tool layer constant so the only variable is the pattern. Then the delta is attributable to the recorded decision.

When to keep it simple

0:25–0:30

The source's advice, turned into ADR-1's null hypothesis

Step	Condition	What it means for ADR-1
Start	A single model call with retrieval and in-context examples. Measure it.	Baseline row in the scorecard
Add a workflow	Only when the task decomposes cleanly and the gain is measurable.	Predicted delta, written before the run
Add an agent	Only when the path cannot be scripted, and trust, budget, and evaluation exist.	The E1 halts and ledger are the precondition
Add a framework	Only when it buys something your stdlib loop cannot: checkpoints, tracing, typed handoffs.	'None / stdlib' is a legitimate alternative and must appear in ADR-1

The abstraction tax: every layer a framework adds is a layer you cannot see into during Drift. If you cannot explain what the framework did on turn three, it owns you.

Teardown: LangChain to LangGraph (2022 to 2024)

0:30–0:35

Why chains became graphs with state

LangChain (late 2022)

Chains, agents, tools, memory as objects: the first wrapper around the ReAct prompt. It made the loop importable, and it hid the loop. Practitioners paid the abstraction tax.

LangGraph (2024)

The agent as a state graph: nodes are steps, edges are control flow you draw, state is typed, checkpoints persist it, a human interrupt is an edge. You own every edge.

Component	LangChain	LangGraph
Planning	Hidden in the agent class; prompt-driven	Explicit graph; conditional edges
Memory & context	Memory objects, opaque	Typed state + checkpointer
Governance & human	None	interrupt() before an edge; resume
Runtime	A script	Persisted runs; streaming; tracing (LangSmith)

Teardowns: Pydantic AI and the role-based frameworks

0:35–0:40

Types as governance; conversation as coordination

Pydantic AI (2024–25): why typed outputs

Every output is validated against a Pydantic model before it acts; malformed ones are retried or rejected.

Theory of control: the type system owns the model–code boundary. E1's Step schema in miniature.

Cost: what the schema cannot express, the model cannot do (E3).

CAMEL · MetaGPT · AutoGen · CrewAI (2023–24)

CAMEL: two role-playing agents. MetaGPT: a software company as prompts, SOPs as coordination. AutoGen: conversation as coordination. CrewAI: roles, goals, backstories, a process.

Theory of control: conversation among personas; FIPA and BDI rediscovered (E8).

Cost: no evaluator; one role's summary becomes the next role's fact.

Teardown: vendor SDKs (2025), who owns the loop?

0:40–0:45

OpenAI Agents SDK · Claude Agent SDK · Google ADK — what each assumes

	OpenAI Agents SDK	Claude Agent SDK	Google ADK
Theory of control	Agents with handoffs; you compose, the runner loops	The harness is the library: the vendor's loop, your tools and instructions	Hierarchy of agents; workflow agents (sequential, parallel, loop) plus LLM agents
Planning	Model-driven within each agent	Claude Code's loop, plan mode, subagents	Explicit workflow agents or model-driven
Tools	Function tools; hosted tools (search, computer)	Built-in file/shell tools + MCP; permissions; hooks	Function tools; MCP; OpenAPI
Coordination	Handoffs between agents	Subagents; Agent tool	Agent trees; A2A for peers
Evaluation	Guardrails (input/output); tracing built in	Hooks; the human per tool call	Evaluation framework; callbacks
Governance & human	Guardrail tripwires	Permission prompts; hooks as membrane	Callbacks; human-in-the-loop via tools
Runtime	Runner; sessions; tracing UI	CLI/SDK; background and cloud sessions	Local dev UI; Vertex deployment

Details as of early 2026; these SDKs change monthly, which is itself the lesson.

Reference design: ADR-0.0.2 — 'popularity is not rationale'

0:45–0:50

The model ADR: argparse over click, and the shape every ADR-1 should take

What the ADR did

Decision: gzkit's CLI on argparse, not click.

Alternatives: click, typer, argparse, with consequences, not adjectives.

Popularity was rejected as a reason. What counted: zero external dependencies, a smaller attack and upgrade surface, stdlib-first posture.

[Summarized from the ADR title; verify against the ADR text.]

The shape of ADR-1

1. Intent (from the PRD)
2. Decision: pattern + framework, one sentence
3. Alternatives: at least three, with 'none / stdlib'
4. Consequences: the one Sortie 2 measures
5. Decomposition scorecard against the alternatives
6. Feature checklist through E12

ADRs before code.

Three forcing functions and ten peer challenges

0:50–0:52

How the reference design stress-tests a decision before it is recorded

Pre-mortem

Week twelve: this framework choice failed. Write the story. What you wrote is a consequence.

WWHTBT

'What would have to be true' for the rejected alternative to be right? If plausible, your decision needs a measurable consequence.

Constraint archaeology

Every constraint you are honoring — where did it come from? PRD, syllabus, habit? Habit is not a constraint.

Red-team a peer's ADR-1 with the ten challenges

1. So what? What changes if wrong
2. Scope: what is out
3. Alternative: the strongest not picked
4. Dependency: what this requires
5. Gold standard: define good
6. Timeline: when we know
7. Evidence: measured, not read
8. Consumer: who lives with it
9. Regression: what got worse
10. Parity: what yours cannot do

Evaluation lens: strong Sortie 2, strong ADR-1

0:52–0:56

The Chase reads the two traces and the ADR's predicted consequence

Weak

- Framework chosen because a tutorial used it
- Alternatives listed as names, no consequences
- Refactor changed the tools too; delta unattributable
- 'Better' asserted; nothing measured
- ADR written after the code

Sound

- Two patterns, same task, identical tool traces in the ledger
- One consequence predicted in ADR-1 and measured
- Three alternatives including none/stdlib, with costs
- Scorecard filled; most features does not win
- ADR committed before the refactor commit

Exceptional

- Predicted consequence absent, and the ADR amended with the evidence
- A pre-mortem naming a week-12 failure and a mitigation
- The peer's ten challenges answered in the ADR
- What the framework hid on turn three, and how you found out

Flight Card 2: two patterns, one trace

0:56–1:00

Center of the envelope · depends on Sortie 1 PASS

ENVELOPE	center
DESIGN CLAIM	Same task, two patterns, identical tool traces, the delta ADR-1 predicted.
SYSTEMS UNDER TEST	E1 loop · pattern A · pattern B · tool layer (constant) · ledger · ADR-1
DEPENDS ON	Sortie 1 PASS
ENTRY CONDITIONS	clean tree · ADR-1 committed BEFORE the refactor · Go recorded
TEST POINTS	1 pattern A → ledger: tool events [$A_1 \ldots A_n$]; measurement
	2 pattern B → ledger: identical tool sequence; measurement
	3 compare → delta as ADR-1 predicted
INSTRUMENTATION	ledger: two runs; receipts: run hashes; ADR-1 dated before TP1
FALSIFIERS	sequences differ · no measurement · predicted delta absent · ADR dated after refactor
ABORT	framework failure unrelated to the pattern (squawk)
CHASE	peer, from two ledger slices and the ADR → PASS / FAIL

A framework is a theory of who owns the loop.

Choose one on evidence, write down what you gave up, and be ready to be wrong in the ledger.